

Índice general

I	Introducción: del cálculo proposicional al código	3
1.	Proposiciones y conectivos (sintaxis)	5
2.	Valoraciones y tablas de verdad (semántica)	6
3.	Consecuencia lógica	7
4.	Cómo decide <code>calcprop</code> : tableaux semánticos	8
II	Aparato lógico	9
5.	Estructura del paquete <code>calcprop</code>	10
6.	Clase <code>Proposicion</code> y su subclase <code>v</code> (proposición atómica)	11
6.1.	Las operaciones: conjunción, disyunción, negación, implicación y equivalencia.	12
6.1.1.	Precedencia de los operadores en Python	12
6.2.	Las operaciones: <code>unoDe</code> y <code>alguno</code>	13
6.3.	La operación: <code>noDerivable</code>	13
6.4.	La subclase <code>v</code> para proposiciones atómicas (i.e., para variables proposicionales)	14
7.	Valoración de proposiciones	15
7.1.	Función <code>span</code>	16
7.1.1.	Implementación	16
8.	Función <code>extension</code> que extiende una función parcial	18
8.1.	El método: tableaux semánticos	18
8.2.	Reducción de proposiciones NO atómicas	19
8.2.1.	Regla de la negación (<code>-</code>)	19
8.2.2.	Regla de la conjunción (<code>&</code>)	19
8.2.3.	Regla de la disyunción (<code> </code>)	19
8.2.4.	Regla de la implicación (<code>>></code>)	19
8.2.5.	Regla de la equivalencia (<code>**</code>)	19
8.2.6.	Regla de <code>alguno</code>	20
8.2.7.	Regla de <code>unoDe</code>	20
8.2.8.	Tabla de reglas	20
8.3.	La traza del razonamiento	20
8.4.	Implementación de la función <code>extension</code>	21
8.4.1.	Proposición atómica (o <i>variable proposicional</i>)	22
8.4.2.	Proposición no derivable de un conjunto de <code>premisas</code>	22
9.	Función que <code>refuta</code> una proposición a partir de un conjunto de <code>premisas</code>	23
10.	Función <code>test</code>	24
11.	Representación de la clase <code>Proposicion</code>	25

III	Herramientas didácticas	26
12.	Átomos y subfórmulas	28
13.	Tablas de verdad	29
14.	Explicación del razonamiento	32

Parte I

Introducción: del cálculo proposicional al código

Este documento es a la vez un texto y un programa: el fichero `src/calccprop/__init__.py` se genera (*tangle*) a partir de los bloques de código que aparecen en él. Cada concepto del cálculo proposicional tiene aquí su definición matemática y, a continuación, la función de Python que lo implementa. Este primer capítulo es un mapa: presenta los conceptos y señala en qué sección se convierte cada uno en código. Para una versión interactiva, con ejemplos ejecutables, véase el cuaderno `Introduccion.ipynb`.

Capítulo 1

Proposiciones y conectivos (sintaxis)

Una *proposición* es una afirmación que es verdadera o falsa, aunque no sepamos cuál de las dos cosas: “llueve”, “la matriz A es invertible”. El cálculo proposicional no mira dentro de las proposiciones; las trata como átomos y solo se ocupa de cómo se combinan mediante *conectivos*:

negación “no P ” (en lógica $\neg P$; en `calccprop`, `-P`),

conjunción “ P y Q ” ($P \wedge Q$; `P & Q`),

disyunción “ P o Q ”, en sentido inclusivo ($P \vee Q$; `P | Q`),

implicación “si P entonces Q ” ($P \rightarrow Q$; `P >> Q`),

equivalencia “ P si y solo si Q ” ($P \leftrightarrow Q$; `P ** Q`).

Una *fórmula* es lo que resulta de combinar átomos con conectivos, respetando las reglas de formación: un átomo es una fórmula, y si P y Q son fórmulas también lo son $\neg P$, $P \wedge Q$, $P \vee Q$, $P \rightarrow Q$ y $P \leftrightarrow Q$. Toda fórmula es un árbol: en cada nodo hay un conectivo y en las hojas, átomos.

En el código, ese árbol es la clase `Proposicion` (sección 6): el atributo `data` es una etiqueta si la proposición es atómica, o una lista `[conectivo, hijo, hijo]` si es compuesta. Los métodos `__and__`, `__or__`, etc. son las reglas de formación. Como los símbolos son los de Python, también lo es su precedencia, que no coincide con la habitual en lógica (sección 6.1.1).

Capítulo 2

Valoraciones y tablas de verdad (semántica)

Fijar un *mundo* es decidir qué átomos son verdaderos y cuáles falsos. Una vez fijado, el valor de cualquier fórmula queda determinado por las tablas de los conectivos: $P \wedge Q$ es verdadera solo si lo son ambas; $P \vee Q$ es falsa solo si lo son ambas; $\neg P$ invierte el valor; $P \rightarrow Q$ es falsa únicamente cuando P es verdadera y Q falsa; $P \leftrightarrow Q$ es verdadera cuando ambas tienen el mismo valor. La función que asigna así un valor a cada fórmula se llama *valoración* (sección 7); en el código es `span(F, P)`, donde `F` es un diccionario que fija el mundo.

Con n átomos hay 2^n mundos posibles. La *tabla de verdad* de una fórmula recorre todos ellos (sección 13, función `tabla_verdad`). Según lo que se observe en la tabla, la fórmula es una *tautología* (verdadera en todos los mundos, como $P \vee \neg P$), una *contradicción* (falsa en todos, como $P \wedge \neg P$) o *contingente* (depende del mundo).

Capítulo 3

Consecuencia lógica

Lo que de verdad nos interesa no es si una fórmula es verdadera, sino si *se deduce* de otras. Se dice que P es *consecuencia lógica* de las premisas $H_1, \dots, H_n$, y se escribe $H_1, \dots, H_n \models P$, cuando en todo mundo en que las premisas son verdaderas también lo es P . Dicho de otro modo: no existe ningún *contraejemplo*, un mundo con premisas verdaderas y P falsa.

Con la tabla de verdad esto se comprueba mirando solo las filas en que todas las premisas valen V, y viendo si en todas ellas P vale V. En el código, `test(P, premisas)` (sección 10) responde exactamente a esa pregunta. Obsérvese que `test` puede devolver `False` por dos razones distintas: porque $\neg P$ se deduzca de las premisas, o porque las premisas no determinen P en absoluto. `noDerivable` (sección 6.3) permite distinguir ambos casos.

Capítulo 4

Cómo decide **calcprop**: tableaux semánticos

Recorrer los 2^n mundos es inviable en cuanto hay unas decenas de átomos. **calcprop** usa en su lugar el método de los *tableaux semánticos* (o *árboles de refutación*), que es lo que hace un lógico con lápiz y papel: para ver si $H \models P$, *supone* que existe un contraejemplo (las premisas son V y P es F) y va descomponiendo esas suposiciones sobre fórmulas compuestas en suposiciones sobre sus partes, según reglas que se leen directamente de las tablas de verdad:

- Si $P \wedge Q$ debe ser V, entonces P debe ser V y Q debe ser V (una sola rama).
- Si $P \wedge Q$ debe ser F, entonces o bien P es F, o bien Q es F (el razonamiento se bifurca en dos ramas).
- Si $P \rightarrow Q$ debe ser V, entonces o bien P es F, o bien Q es V; si debe ser F, entonces P es V y Q es F.
- Y así con cada conectivo (sección 8.2).

Cuando una suposición llega a un átomo, se *anota* su valor. Si más adelante la misma rama exige el valor contrario para ese átomo, la rama *se cierra*: era imposible. Si todas las ramas se cierran, no existe contraejemplo y por tanto $H \models P$. Si alguna rama termina sin contradicción, sus anotaciones son precisamente un contraejemplo, y P no se deduce.

La función **extension** (sección 8) es este procedimiento; **refuta** lo pone en marcha con las suposiciones iniciales y **test** interpreta el resultado. La función **explica** (sección ??) muestra el árbol paso a paso, tal como se escribiría a mano.

Parte II

Aparato lógico

Capítulo 5

Estructura del paquete `calcprop`

El fichero `src/calcprop/__init__.py` se genera (*tangle*) a partir de este documento. Comienza con la cabecera del módulo y la lista `__all__` de nombres públicos, que es lo que importa `from calcprop import *`.

```
"""calcprop: un motor de cálculo proposicional.

Las fórmulas se construyen con ``v('etiqueta)`` (proposición atómica) y los
operadores ``&`` (y), ``|`` (o), ``-`` (no), ``>`` (implica) y ``*`` (equivale),
más las funciones ``alguno``, ``unoDe`` y ``noDerivable``. La función ``test(P,
premisas)`` decide si ``P`` es consecuencia lógica de las ``premisas``;
``explica`` muestra el razonamiento paso a paso y ``tabla_verdad`` construye
la tabla de verdad.
"""

import html as _html
import itertools as _itertools

__all__ = ['Proposicion', 'v', 'alguno', 'unoDe', 'noDerivable',
           'span', 'extension', 'refuta', 'test',
           'atomos', 'subformulas', 'tabla_verdad', 'TablaVerdad',
           'explica', 'Razonamiento']

<<Aparato lógico de la librería>>
```

```
<<Definición de la clase Proposicion>>
<<Definición de la subclase v>>
<<Definición de alguno y unoDe>>
<<Definición de noDerivable>>
<<Función auxiliar contiene>>
<<Definición de la función span>>
<<Reglas de reducción>>
<<Funciones auxiliares de la traza>>
<<Definición de la función extension>>
<<Definición de la función refuta>>
<<Definición de la función test>>
<<Átomos y subfórmulas>>
<<Tabla de verdad>>
<<Explicación del razonamiento>>
```

El aparato auxiliar para la creación de problemas de opción múltiple (clases `Supuesto`, `Cuestion`, `ProblemaTipo`, exportación a Moodle y AMC, etc.) se desarrolla en el paquete `qbank`, que usa `calcprop` como motor lógico.

Capítulo 6

Clase `Proposicion` y su subclase `v` (proposición atómica)

Las proposiciones son expresiones formadas con números, strings y los símbolos

`&`, `|`, `-`, `v( )`, `>>`, `**`, `alguno` y `unoDe`

construidas según las siguientes reglas:

1. Si `D` es un número o un string (cadena de caracteres) entonces `v(D)` es una subclase de `Proposicion` que denominaremos “*variable proposicional*” o “*proposición atómica*”.
2. Si `P`, `Q` y `R` son proposiciones también lo son:

`P&Q` (conjunción)

`P|Q` (disyunción)

`-P` (negación)

`P>>Q` (implicación)

`P**Q` (equivalencia, es decir: *si y solo si*).

Para facilitar la elaboración de expresiones, también incluimos dos operadores lógicos menos habituales:

`alguno(P,Q,R)` significa que alguna de las proposiciones de la lista es verdadera y

`unoDe(P,Q,R)` significa que solo una de las proposiciones de la lista es verdadera.

Ejemplo de proposición: “llueve o no llueve”

```
v('llueve') | -v('llueve')
```

```
v('llueve') | -v('llueve')
```

Necesitamos que las proposiciones sean “*hash-eables*”, para poder ser utilizadas como claves en un diccionario. Para ello necesitamos los métodos `__hash__` y `__eq__`. Las proposiciones atómicas se comparan y se *hashean* por su etiqueta `data`; las compuestas, cuyo `data` es una lista (y las listas no son *hasheables*), usan su representación textual, que determina unívocamente la fórmula.

Además definimos `__bool__` para que lance un error. El motivo es que Python no permite redefinir `and`, `or` y `not`: si un usuario escribe `v('A') and v('B')` obtiene `v('B')`, y si escribe `not v('A')` obtiene `False`, en ambos casos sin ningún aviso. Con `__bool__` lanzando `TypeError` ese error se detecta en el acto, con un mensaje que indica que hay que usar `&`, `|` y `-`.

Definamos la clase `Proposicion`

```

class Proposicion:
    """Fórmula del cálculo proposicional.

    ``data`` es una etiqueta (número o string) si la proposición es atómica
    (véase la subclase ``v``), o una lista ``[operador, operando, ...]`` si es
    compuesta, con ``operador`` en ``'and'``, ``'or'``, ``'not'``, ``'implica'``,
    ``'equivale'``, ``'alguno'``, ``'unoDe'`` o ``'noDerivable'``.

    Las fórmulas se combinan con los operadores ``&``, ``|``, ``-``, ``>>`` y
    ``***`` (nunca con ``and``, ``or`` y ``not``, que Python no permite redefinir).
    """
    def __init__(self, data):
        self.data = data

    def __hash__(self):
        if isinstance(self.data, list):
            return hash(repr(self))
        return hash(self.data)

    def __eq__(self, another):
        return isinstance(another, Proposicion) and self.data == another.data

    def __bool__(self):
        raise TypeError(
            "Una Proposicion no tiene valor de verdad por sí misma: use "
            "test(P, premisas). Para combinar proposiciones use los operadores "
            "&, | y - en lugar de and, or y not.")

<<Operadores lógicos de la clase Proposicion>>
<<Método de representación de la clase Proposicion>>

```

6.1. Las operaciones: conjunción, disyunción, negación, implicación y equivalencia.

Definamos como métodos dentro de la clase `Proposicion` los cinco operadores lógicos habituales. Primero aquellos que usan “*métodos mágicos*”, es decir, aquellos que tiene asociado un símbolo para llamar a las operaciones:

```

def __and__(self, other):
    return Proposicion(['and', self, other])
def __or__(self, other):
    return Proposicion(['or', self, other])
def __neg__(self):
    return Proposicion(['not', self])
def __rshift__(self, other):
    return Proposicion(['implica', self, other])
def __pow__(self, other):
    return Proposicion(['equivale', self, other])

```

6.1.1. Precedencia de los operadores en Python

Al reutilizar operadores de Python heredamos su precedencia, que **no coincide** con la habitual en lógica. De mayor a menor prioridad, Python evalúa:

1. `**` (equivalencia),
2. `-` unario (negación),
3. `>>` (implicación),
4. `&` (conjunción),
5. `|` (disyunción).

Las consecuencias más traicioneras son:

- `A & B >> C` se lee como `A & (B >> C)`, no como `(A & B) >> C`.
- `A | B >> C` se lee como `A | (B >> C)`.
- `-A ** B` se lee como `-(A ** B)`, no como `(-A) ** B`, porque `**` liga más fuerte que el menos unario.

- $A \gg B \gg C$ se lee como $(A \gg B) \gg C$, mientras que en lógica lo habitual es leer $A \gg (B \gg C)$.

La recomendación es sencilla: **siempre que se mezclen conectivos distintos, poner paréntesis**. Solo $\neg A \& B$ y $\neg A \mid B$ son seguros sin paréntesis (el menos unario liga más fuerte que $\&$ y $\mid$).

```
A, B, C = v('A'), v('B'), v('C')
print( (A & B >> C) == (A & (B >> C)) )
print( (~A ** B) == ~(A ** B) )
```

True

True

6.2. Las operaciones: unoDe y alguno.

Para las dos siguientes operaciones¹ no usamos ningún “*método mágico*”. Así que definir las dentro de la clase nos obligaría a una incómoda escritura del tipo: `Proposicion.alguno(P1,P2,P3)` o `Proposicion.unoDe(P1,P2,P3)`. Para evitarlo, las definimos fuera de la clase; así podremos escribir sencillamente `alguno(P1,P2,P3)` y `unoDe(P1,P2,P3)`.

```
def alguno(X, *args):
    """alguno(A, B, ...): al menos una de las proposiciones es verdadera."""
    return Proposicion(['alguno', X] + list(args))
def unoDe(X, *args):
    """unoDe(A, B, ...): exactamente una de las proposiciones es verdadera."""
    return Proposicion(['unoDe', X] + list(args))
```

6.3. La operación: noDerivable.

Esta función no es una operación lógica (como sí lo son las anteriores). Es un “añadido” cuya motivación es la siguiente:

El esquema típico de una pregunta es: el enunciado asume que las premisas $E_1, E_2, \dots$ son ciertas. Y se puede preguntar si P es consecuencia lógica de las premisas. Si lo es, P es True (es decir, $\text{premisas} \Rightarrow P$). Negar esto no es decir que $\neg P$ es consecuencia de las premisas, sino que P no es derivable de las premisas (es decir, $\text{premisas} \not\Rightarrow P$). Así que añadimos la operación `noDerivable`. Esto nos permite, por ejemplo, que aunque para algunos enunciados se pueda preguntar si una matriz es diagonalizable, para otros evitemos la pregunta si no es deducible del enunciado.

`noDerivable(P)` solo tiene sentido en la fórmula que se somete a `test`, nunca dentro de las premisas (sería una afirmación *sobre* las premisas, no una premisa). La función `refuta` lo comprueba y lanza un error si ocurre.

```
def noDerivable(X):
    """noDerivable(P): ``P`` no es consecuencia lógica de las premisas.

    Solo tiene sentido en la fórmula que se evalúa con ``test``; no puede
    aparecer dentro de las premisas.
    """
    return Proposicion(['noDerivable', X])
```

Nos será útil una pequeña función auxiliar que diga si un operador aparece en algún lugar de una fórmula:

```
def _contiene(P, operador):
    """True si el ``operador`` aparece en algún subárbol de la fórmula ``P``."""
    if not isinstance(P, Proposicion) or not isinstance(P.data, list):
        return False
    return P.data[0] == operador or any(_contiene(x, operador) for x in P.data[1:])
```

¹no habituales, aunque convenientes pues facilitan la expresión de algunas proposiciones

6.4. La subclase `v` para proposiciones atómicas (i.e., para variables proposicionales)

```
class v(Proposicion):
    """Proposición atómica (variable proposicional) con etiqueta ``data``.

    Dos llamadas ``v('A')`` producen proposiciones iguales entre sí (misma
    etiqueta), de modo que pueden usarse como claves de un diccionario.
    """
    pass
```

Capítulo 7

Valoración de proposiciones

Una valoración es una función T que va del conjunto de proposiciones al conjunto $\{\text{True}, \text{False}\}$ (es decir, a cada proposición le asigna el valor verdadero o el valor falso) y que verifica las siguientes propiedades: si P y Q son proposiciones entonces

1. $T(P \ \& \ Q) = \text{True}$ únicamente en el caso de que $T(P) = \text{True}$ y $T(Q) = \text{True}$.
2. $T(P \ | \ Q) = \text{False}$ únicamente en el caso de que $T(P) = \text{False}$ y $T(Q) = \text{False}$.
3. $T(\neg P) = \text{True}$ únicamente en el caso de que $T(P) = \text{False}$.

Por tanto: $T(P \ \& \ Q) = T(P) \ \& \ T(Q)$, $T(P \ | \ Q) = T(P) \ | \ T(Q)$ y $T(\neg P) = \text{not } T(P)$.

1. $T(P \ \>> \ Q) = \text{False}$ únicamente en el caso en que $T(P) = \text{True}$ y $T(Q) = \text{False}$.
2. $T(P \ ** \ Q) = \text{True}$ únicamente en el caso en que $T(P) = T(Q)$.

Por tanto $T(P \ \>> \ Q) = T(\neg P \ | \ Q)$ y $T(P \ ** \ Q) = T(P \ \>> \ Q) \ \& \ (Q \ \>> \ P)$.

Y si $A_1, \dots, A_n$ son n proposiciones, entonces

1. $\text{alguno}(A_1, \dots, A_n)$ es True si y solo si $A_1 \ | \ \text{alguno}(A_2, \dots, A_n)$ es True ,
2. $\text{unoDe}(A_1, \dots, A_n)$ es True si y solo si $(A_1 \ \& \ \neg \text{alguno}(A_2, \dots, A_n)) \ | \ (\neg A_1 \ \& \ \text{unoDe}(A_2, \dots, A_n))$ es True ;

Por tanto

$$T(\text{alguno}(A_1, \dots, A_n)) = T(A_1 \ | \ \text{alguno}(A_2, \dots, A_n))$$

y

$$T(\text{unoDe}(A_1, \dots, A_n)) = T((A_1 \ \& \ \neg \text{alguno}(A_2, \dots, A_n)) \ | \ (\neg A_1 \ \& \ \text{unoDe}(A_2, \dots, A_n)))$$

Cualquier función F que vaya del conjunto de proposicionales atómicas (es decir, del tipo $v(\text{'algo'})$) al conjunto $\{\text{True}, \text{False}\}$ se puede extender al resto proposiciones¹ formando una valoración $T = \text{span}(F)$, aplicando las reglas de arriba. Dicha extensión es única.

Vamos a implementar como funciones los diccionarios de Python, pues los diccionarios son una colección de pares `'Etiqueta':valor` donde a cada etiqueta solo se le asigna un valor único. Por ejemplo, si definimos el siguiente diccionario `{'a':1, 'a':0, 'b':2}`, obtenemos un diccionario con dos elementos, donde a `'a'` se le ha asignado solo uno de los dos valores introducidos (en concreto el último).

```
print( {'a':1, 'a':0, 'b':2} )
```

```
{'a': 0, 'b': 2}
```

¹En la función `span` de Python hemos elegido el criterio de que a toda proposición `Prop` no incluida explícitamente en el diccionario `F` le asignamos el valor `False` para así completar el dominio de las proposiciones atómicas. Esta decisión ha sido completamente arbitraria, y se ha incluido para que efectivamente el dominio de `span` incluya todas las proposiciones atómicas (i.e., variables proposicionales). `span` no interviene en `test`; se usa para construir tablas de verdad.

7.1. Función span

Función `span(F, Prop)` indica si `Prop` es `True` o es `False` a partir de la función parcial `F`.

- `F`: es un diccionario (función parcial) que asigna valores `True` o `False` a varias proposiciones atómicas.
- `Prop`: es una `Proposicion`.

Así obtenemos los siguientes resultados lógicos:

```
F = { v('llueve'):True }  
span(F, v('llueve') >> -v('llueve') )
```

False

y por otra parte

```
span(F, -v('llueve') >> v('llueve') )
```

True

7.1.1. Implementación

Si una proposición `Prop` (de tipo atómico) está en la función parcial `F` y además es verdadera, `span` devuelve `True`. En caso contrario devuelve `False` (tanto si `Prop` no está en `F` como si lo está pero es `False`). Si `Prop` no es de tipo atómico, entonces se aplican las reglas para reducir `Prop` a proposiciones de tamaño más pequeño. `noDerivable` no es evaluable por una valoración (depende de las premisas, no de los valores de los átomos), así que `span` lanza un error si lo encuentra; lo mismo hace ante un operador desconocido.

```
def span(F, Prop):  
    """Valor de verdad de ``Prop`` bajo la valoración parcial ``F``.  
  
    ``F`` es un diccionario ``{v('A'): True, v('B'): False, ...}``; toda  
    proposición atómica ausente de ``F`` se considera ``False``.  
    """  
    if not isinstance(Prop.data, list):  
        return bool(Prop in F and F[Prop])  
    <<Aplicación de las reglas para reducir el tamaño de Prop>>
```

```
op = Prop.data[0]  
if op == 'and':  
    return span(F, Prop.data[1]) and span(F, Prop.data[2])  
  
elif op == 'or':  
    return span(F, Prop.data[1]) or span(F, Prop.data[2])  
  
elif op == 'not':  
    return not span(F, Prop.data[1])  
  
elif op == 'implica':  
    return (not span(F, Prop.data[1])) or span(F, Prop.data[2])  
  
elif op == 'equivale':  
    return span(F, Prop.data[1]) == span(F, Prop.data[2])  
  
elif op == 'alguno':  
    A = Prop.data[1]  
    if len(Prop.data) == 2:  
        return span(F, A)  
    else:  
        B = Proposicion(['alguno'] + Prop.data[2:])  
        return span(F, A | B)  
  
elif op == 'unoDe':  
    A = Prop.data[1]  
    if len(Prop.data) == 2:  
        return span(F, A)  
    else:  
        B = Proposicion(['alguno'] + Prop.data[2:])  
        C = Proposicion(['unoDe'] + Prop.data[2:])  
        return span(F, (A & -B) | (-A & C))
```

```
elif op == 'noDerivable':  
    raise ValueError("span no puede evaluar noDerivable: depende de las "  
        "premisas, no de una valoración; use test(P, premisas).")  
else:  
    raise ValueError(f"Operador desconocido: {op!r}")
```

Capítulo 8

Función **extension** que extiende una función parcial

Ahora queremos definir una función de Python (que denominamos **extension**) que, dadas una lista **objetivo** y un diccionario con **valores**, para una colección de proposiciones atómicas haga crecer (extienda) el diccionario **valores** con nuevas proposiciones atómicas, y sus correspondientes valores, necesarios para que se cumplan los pares contenidos en la lista **objetivo**; o bien que devuelva el diccionario vacío {} en caso de que sea imposible hacer cumplir dicha lista **objetivo**.

La diferencia entre la lista **objetivo** y los **valores** es que en **objetivo** se asignan valores a proposiciones (arbitrarias) pero en **valores** solo se asignan valores a *proposiciones atómicas*.

El diccionario extendido es un *modelo lógico* (un *mundo*) en el que se cumplen la lista **objetivo** y la función parcial **valores**. Ello no quiere decir que no puedan existir otros modelos que consigan lo mismo.

Ejemplo:

```
print( extension( [ (v('llueve') >> -v('llueve')) , True] , {v('basilisco') : True} ) )
```

```
{v('basilisco'): True, v('llueve'): False}
```

Ejemplo:

```
print( extension( [ (v('llueve'),True), (-v('llueve'),True)], {v('basilisco') : True} ) )
```

```
{}
```

8.1. El método: tableaux semánticos

extension implementa el método de los *tableaux semánticos*. Cada par (P, VF) del **objetivo** es una *suposición*: “=P= vale VF”, y el procedimiento intenta construir un mundo en el que todas las suposiciones se cumplan. La mecánica es recursiva: se toma la primera suposición y

- si P es un átomo, se anota su valor en **valores** (o se comprueba que coincide con lo ya anotado; si no coincide, la rama *se cierra*: es imposible);
- si P es compuesta, una *regla de reducción* la sustituye por suposiciones sobre sus partes. Algunas reglas producen una única lista de suposiciones nuevas (por ejemplo, “=A & B= es V” se convierte en “=A= es V y B es V”); otras *bifurcan* el razonamiento en varias ramas alternativas (“=A & B= es F” se convierte en “o bien A es F, o bien B es F”), y basta con que una de ellas prospere.

Este procedimiento reduce las proposiciones hasta obtener proposiciones atómicas. Es correcto y completo para la lógica proposicional, pero de coste exponencial en el peor caso (para las fórmulas cortas de un banco de preguntas es irrelevante).

8.2. Reducción de proposiciones NO atómicas

Cada regla es una función que recibe la proposición P y el valor VF que debe tomar, y devuelve la lista de *ramas*; cada rama es a su vez una lista de pares (*subfórmula*, *valor*) que deben cumplirse simultáneamente. Las reglas se leen directamente de las tablas de verdad de los conectivos (sección 7).

8.2.1. Regla de la negación (-)

$\neg A$ es V exactamente cuando A es F, y viceversa: una sola rama.

```
def _regla_not(P, VF):
    A = P.data[1]
    return [(A, not VF)]
```

8.2.2. Regla de la conjunción (&)

Si $A \& B$ es V, entonces A es V y B es V (una rama con dos suposiciones). Si $A \& B$ es F, entonces o bien A es F, o bien B es F (dos ramas).

```
def _regla_and(P, VF):
    A, B = P.data[1], P.data[2]
    if VF:
        return [(A, True), (B, True)]
    return [(A, False), (B, False)]
```

8.2.3. Regla de la disyunción (|)

Es la imagen especular de la anterior: si $A | B$ es F, entonces A es F y B es F; si $A | B$ es V, o bien A es V, o bien B es V.

```
def _regla_or(P, VF):
    A, B = P.data[1], P.data[2]
    if VF:
        return [(A, True), (B, True)]
    return [(A, False), (B, False)]
```

8.2.4. Regla de la implicación (>>)

$A \gg B$ solo es F cuando A es V y B es F (una rama). Si es V, entonces o bien A es F, o bien B es V (dos ramas); es la regla de $\neg A | B$.

```
def _regla_implica(P, VF):
    A, B = P.data[1], P.data[2]
    if VF:
        return [(A, False), (B, True)]
    return [(A, True), (B, False)]
```

8.2.5. Regla de la equivalencia (**)

$A ** B$ es V cuando ambas valen lo mismo: o bien las dos V, o bien las dos F. Es F cuando valen distinto: o bien A es V y B es F, o bien al revés. En ambos casos hay dos ramas.

```
def _regla_equivale(P, VF):
    A, B = P.data[1], P.data[2]
    if VF:
        return [(A, True), (B, True), (A, False), (B, False)]
    return [(A, True), (B, False), (A, False), (B, True)]
```

8.2.6. Regla de alguno

$\text{alguno}(A_1, \dots, A_n)$ es V si alguna A_i es V: una rama por alternativa. Es F si todas son F: una sola rama con n suposiciones.

```
def _regla_alguno(P, VF):
    alternativas = P.data[1:]
    if VF:
        return [(A, True)] for A in alternativas
    return [(A, False)] for A in alternativas
```

8.2.7. Regla de unoDe

$\text{unoDe}(A_1, \dots, A_n)$ es V si exactamente una A_i es V: hay n ramas, y en la rama i se supone A_i V y todas las demás F. Es F si ninguna es V o si al menos dos lo son: una rama con todas F, más una rama por cada par (A_i, A_j) con ambas V.

```
def _regla_unoDe(P, VF):
    alternativas = P.data[1:]
    if VF:
        return [(Aj, j == i) for j, Aj in enumerate(alternativas)
                for i in range(len(alternativas))]
    ramas = [(A, False) for A in alternativas]
    for i, j in itertools.combinations(range(len(alternativas)), 2):
        ramas.append([(alternativas[i], True), (alternativas[j], True)])
    return ramas
```

8.2.8. Tabla de reglas

Reunimos las reglas en un diccionario indexado por el nombre del operador, junto con el símbolo con que se mostrará cada regla en la traza del razonamiento.

```
<<Regla not>>
<<Regla and>>
<<Regla or>>
<<Regla implica>>
<<Regla equivale>>
<<Regla alguno>>
<<Regla unoDe>>
_REGLAS = {'not': _regla_not, 'and': _regla_and, 'or': _regla_or,
            'implica': _regla_implica, 'equivale': _regla_equivale,
            'alguno': _regla_alguno, 'unoDe': _regla_unoDe}
_NOMBRE_REGLA = {'not': '-', 'and': '&', 'or': '|', 'implica': '>>',
                 'equivale': '***', 'alguno': 'alguno', 'unoDe': 'unoDe'}
```

8.3. La traza del razonamiento

Para poder *ver* el razonamiento, *extension* acepta un argumento opcional *traza*: una lista a la que va añadiendo, como texto, cada paso que da, con una sangría proporcional a la profundidad de la rama. Si *traza* es None (el valor por defecto) no se anota nada y no hay coste alguno. Los valores de verdad se muestran como V y F.

```
def _VF(b):
    return 'V' if b else 'F'

def _fmt_suposicion(par):
    return f"{par[0]!r} = {_VF(par[1])}"

def _fmt_ramas(ramas):
    partes = [" y ".join(_fmt_suposicion(s) for s in rama) for rama in ramas]
    if len(partes) == 1:
        return partes[0]
    return "o bien " + ", o bien ".join(partes)

def _es_pseudoatomo(A):
    return isinstance(A.data, str) and A.data.startswith('noDerivable(')

def _fmt_modelo(valores):
    pares = [f"{a!r} = {_VF(b)}" for a, b in valores.items() if not _es_pseudoatomo(a)]
```

```

return "{ " + " ".join(pares) + " }" if pares else "{}"

def _anota(traza, nivel, texto):
    if traza is not None:
        traza.append(" " * nivel + texto)

```

8.4. Implementación de la función `extension`

Función `extension(objetivo, valores, premisas=[], traza=None, nivel=0)`

- **objetivo**: es una lista de pares (P,VF) donde P es una proposición y VF es True o False
- **valores**: es un diccionario (función parcial) que asigna valores True o False a variables proposicionales
- **premisas**: es la lista de proposiciones que se asumen ciertas, a partir de la cual se decide si una proposición es derivable o no (es decir, solo se usa para la operación `noDerivable`)
- **traza**: lista en la que anotar los pasos (o None), y **nivel**: sangría actual de la traza.

La función `extension` tiene dos posibles salidas: o bien devuelve un modelo lógico M que extiende a valores de tal forma que $\text{span}(M,P)=VF$ para cada par (P,VF) de `objetivo`, o el diccionario vacío {} si no existe tal modelo M.

Obsérvese que el diccionario vacío significa “imposible”, y no “modelo que no asigna nada”. No hay ambigüedad porque toda proposición contiene al menos un átomo, de modo que cualquier modelo que satisfaga un objetivo no vacío asigna al menos un valor. El único caso que no toca ningún átomo es `noDerivable`, y por eso en su regla se añade al modelo un *pseudo-átomo* (véase más abajo).

Si la lista `objetivo` no tiene pares, con la propia función `valores`, hemos acabado. Si no, llamamos P a la proposición de la primera suposición y VF a su valor, y distinguimos tres casos: átomo, `noDerivable` y proposición compuesta ordinaria. En este último caso se aplica la regla correspondiente y se prueban sus ramas en orden: la primera que prospere da el resultado; si todas se cierran, se devuelve {}.

```

def extension(objetivo, valores, premisas=[], traza=None, nivel=0):
    """Extiende la valoración parcial ``valores`` hasta satisfacer ``objetivo``.

    ``objetivo`` es una lista de pares ``(P, VF)``; ``premisas`` es la lista de
    proposiciones asumidas ciertas (solo la usa ``noDerivable``). Devuelve un
    modelo (diccionario átomo -> bool) que extiende ``valores`` y en el que cada
    ``P`` toma el valor ``VF``, o ``{}`` si no existe tal modelo. Si ``traza``
    es una lista, se le añaden los pasos del razonamiento como texto.
    """
    if not objetivo:
        _anota(traza, nivel, "✓ No quedan suposiciones por cumplir: el mundo "
              + _fmt_modelo(valores) + " las satisface todas.")
        return valores

    (P, VF), resto = objetivo[0], objetivo[1:]
    if not isinstance(P.data, list):
        <<Proposición atómica v>>
        op = P.data[0]
        if op == 'noDerivable':
            <<Proposición de tipo "noDerivable">>
        if op not in _REGLAS:
            raise ValueError(f"Operador desconocido: {op!r}")
        ramas = _REGLAS[op](P, VF)
        _anota(traza, nivel, f"{_fmt_suposicion((P, VF))} → por la regla de "
              f"{_NOMBRE_REGLA[op]}: {_fmt_ramas(ramas)}")
        for k, rama in enumerate(ramas, 1):
            subnivel = nivel
            if len(ramas) > 1:
                _anota(traza, nivel + 1, f"Rama {k} de {len(ramas)}: suponemos "
                      + " y ".join(_fmt_suposicion(s) for s in rama))
                subnivel = nivel + 2
            M = extension(rama + resto, valores, premisas, traza, subnivel)
            if M:
                return M
        if len(ramas) > 1:
            _anota(traza, nivel + 1, f"× Las {len(ramas)} ramas se cierran.")
        return {}

```

8.4.1. Proposición atómica (o *variable proposicional*)

En el caso de que $P = v(D)$, es cuando el algoritmo recursivo pasará a intentar la siguiente suposición (extendiendo el diccionario `valores` si fuera necesario), o bien parará por ser la suposición $(v(D), VF)$ imposible.

Primero se comprueba si $v(D)$ ya estaba en el diccionario `valores`. Si estaba con el mismo valor `VF`, se “salta” a la siguiente suposición. Si estaba **con un valor distinto**, la rama se cierra: se devuelve un diccionario vacío. Si no estaba, creamos un nuevo diccionario extendido (`valoresExt`) que incluya P y su valor `VF`, y continuamos con el resto. Para evitar los efectos colaterales de modificar un diccionario, modificamos una copia.

```
if P in valores:
    if valores[P] == VF:
        _anota(traza, nivel, f"_{fmt_suposicion((P, VF))} ✓ ya estaba anotado.")
        return extension(resto, valores, premisas, traza, nivel)
    _anota(traza, nivel, f"_{fmt_suposicion((P, VF))} ✗ contradice lo anotado "
            f"({P!r} = {_VF(valores[P])}): rama cerrada.")
    return {}
valoresExt = valores.copy()
valoresExt[P] = VF
_anota(traza, nivel, f"_{fmt_suposicion((P, VF))} → anotamos {P!r} = {_VF(VF)}.")
return extension(resto, valoresExt, premisas, traza, nivel)
```

8.4.2. Proposición no derivable de un conjunto de premisas

El tercer argumento de `extension` (`premisas`) es un “añadido” para decidir si una proposición es `noDerivable` de la lista de premisas.

Derivable significa que la proposición es consecuencia lógica de las `premisas`, por consiguiente es imposible que sea falsa si las `premisas` son ciertas. Así que A es derivable exactamente cuando `refuta(A, premisas)` (definida más abajo) devuelve el diccionario vacío. Es un *subproblema* independiente: su traza se anota con una sangría adicional.

El valor de `noDerivable(A)` no depende de la rama actual del *tableau* (de `valores`), sino solo de las premisas. Por eso, si el valor calculado coincide con el objetivo `VF` se sigue con el resto de suposiciones, y si no coincide la rama se cierra. Para seguir, añadimos al modelo un *pseudo-átomo* $v(\text{"noDerivable(A)"})$ con valor `VF`. Es necesario porque `noDerivable` no toca ningún átomo real: sin el pseudo-átomo, refutar `noDerivable(A)` a solas devolvería el modelo `{}`, que se confundiría con “imposible de refutar”.

```
A = P.data[1]
_anota(traza, nivel, f"_{fmt_suposicion((P, VF))} → ¿se deduce {A!r} de las "
        "premisas? Lo comprobamos aparte:")
subtraza = [] if traza is not None else None
noDerivableA = bool(refuta(A, premisas, subtraza))
if traza is not None:
    traza.extend(" " * (nivel + 1) + linea for linea in subtraza)
_anota(traza, nivel + 1, f"Por tanto {A!r} {'NO' if noDerivableA else 'SÍ'} se deduce "
        f"de las premisas, y {P!r} = {_VF(noDerivableA)}.")
if noDerivableA != VF:
    _anota(traza, nivel, f"✗ {fmt_suposicion((P, VF))} es imposible: rama cerrada.")
    return {}
valoresExt = valores.copy()
valoresExt[v(repr(P))] = VF # pseudo-átomo: el modelo nunca queda vacío
return extension(resto, valoresExt, premisas, traza, nivel)
```

Capítulo 9

Función que refuta una proposición a partir de un conjunto de premisas

Refutar una proposición, consiste en asignar un valor a cada variable proposicional (cada proposición atómica), de tal modo que el resultado sea falso.

La función `refuta` construye un modelo `M` (asignación de valores a las proposiciones atómicas) de forma que `span(M,P)=False` y `span(M,h)=True` para todo `h` en `premisas`.

Si no es posible refutar `P` a partir de las `premisas`, entonces la función `refuta` devuelve un modelo (un diccionario) vacío; esto quiere decir que la proposición `P` es irrefutable, es decir, `premisas  $\Rightarrow$  P`. Es decir, si se refuta es que no es derivable de las premisas (no es consecuencia lógica de las premisas).

Antes de construir el *tableau*, `refuta` sanea la lista de premisas:

- Comprueba que `premisas` es realmente una lista (el despiste habitual es escribir `test(P, Q)` en lugar de `test(P, [Q])`).
- Las premisas `True` se descartan (no aportan información).
- Si alguna premisa es `False`, de una contradicción se deriva cualquier cosa, así que nada es refutable: se devuelve `{}` directamente.
- Si alguna premisa contiene `noDerivable`, se lanza un error: no es una premisa, sino una afirmación sobre las premisas, y además provocaría una recursión infinita.

```
def refuta(P, premisas=[], traza=None):
    """Modelo que hace falsa ``P`` y ciertas las ``premisas``, o ``{}`` si no existe.

    Un resultado ``{}`` significa que ``P`` es consecuencia lógica de las
    ``premisas``. Las premisas ``True`` se ignoran; una premisa ``False`` hace
    que nada sea refutable. Si ``traza`` es una lista, se le añaden los pasos.
    """
    if isinstance(premisas, (Proposicion, bool)):
        raise TypeError("premisas debe ser una lista de proposiciones, "
                        f"por ejemplo test(P, [{premisas!r}]).")
    premisas = [Q for Q in premisas if Q is not True]
    if any(Q is False for Q in premisas):
        _anota(traza, 0, "Una de las premisas es False: de una contradicción se "
                  "deduce cualquier cosa, así que no hay contraejemplo posible.")
        return {}
    for Q in premisas:
        if _contiene(Q, 'noDerivable'):
            raise ValueError("noDerivable no puede aparecer en las premisas "
                            f"(premise: {Q!r}).")
    suposiciones = [(P, False)] + [(Q, True) for Q in premisas]
    _anota(traza, 0, "Buscamos un mundo en el que "
          + ", ".join(_fmt_suposicion(s) for s in suposiciones) + ":")
    return extension(suposiciones, {}, premisas, traza)
```

Capítulo 10

Función `test`

Función `test(P,premisas)`

- `P`: es una proposición o cualquiera de los valores `True`, `False`.
- `premisas`: es una lista de proposiciones (que también puede contener los valores `True` o `False`).

La función `test` se utiliza tanto para saber si se da una precondition, como para saber el resultado correcto de una cuestión.

En el caso de que `P` sea una proposición, sólo devuelve `True` si es imposible refutarla a partir de las `premisas`, es decir, es imposible que `P` sea falsa si las `premisas` son verdad ($\text{premisas} \Rightarrow P$, i.e., `P` es derivable). Si `P` es directamente `True` o `False`, se devuelve tal cual, sin mirar las `premisas`.

```
def test(P, premisas=[]):  
    """¿Es ``P`` consecuencia lógica de las ``premisas``?  
  
    ``P`` puede ser una ``Proposicion`` o directamente ``True``/``False`` (que  
    se devuelven tal cual). ``premisas`` es una lista de proposiciones.  
    """  
    if isinstance(P, bool):  
        return P  
    return not refuta(P, premisas)
```

Capítulo 11

Representación de la clase Proposicion

La representación textual de una proposición debe cumplir una propiedad esencial: ha de ser código Python que, evaluado, reconstruya exactamente la misma proposición ($\text{eval}(\text{repr}(P)) = P$). El paquete `qbank` se apoya en ello para guardar y recuperar las fórmulas en ficheros JSON.

Dado que la precedencia de los operadores de Python no es la de la lógica (véase más arriba), no basta con concatenar los operandos con el símbolo del operador: `repr((A | B) >> C)` daría `v('A') | v('B') >> v('C')`, que Python vuelve a leer como `A | (B >> C)`. Por ello, todo operando que sea a su vez una operación binaria se escribe entre paréntesis, y también una negación cuando es operando de `**` (porque `-A ** B` se lee como `-(A ** B)`). Los átomos, las negaciones de átomos y las llamadas a `alguno`, `unoDe` y `noDerivable` no necesitan paréntesis. Además de correcta, esta representación es más legible para una persona, que tampoco tiene por qué recordar la tabla de precedencias de Python.

```
_SIMBOLO = {'and': '&', 'or': '|', 'implica': '>>', 'equivale': '**'}

def __repr__(self):
    d = self.data
    if not isinstance(d, list):
        return 'v(' + repr(d) + ')'
    op = d[0]

    def operando(x):
        s = repr(x)
        if isinstance(x, Proposicion) and isinstance(x.data, list):
            binaria = x.data[0] in Proposicion._SIMBOLO
            negacion = x.data[0] == 'not' and op == 'equivale'
            if binaria or negacion:
                return '(' + s + ')'
        return s

    if op == 'not':
        return '-' + operando(d[1])
    if op in Proposicion._SIMBOLO:
        return operando(d[1]) + ' ' + Proposicion._SIMBOLO[op] + ' ' + operando(d[2])
    if op in ('alguno', 'unoDe', 'noDerivable'):
        return op + '(' + ', '.join(repr(x) for x in d[1:]) + ')'
    raise ValueError(f"Operador desconocido: {op!r}")
```

```
A, B, C = v('A'), v('B'), v('C')
print( (A | B) >> C )
print( -(A & B) )
print( (-A) ** B )
print( eval(repr((A | B) >> C)) == ((A | B) >> C) )
```

```
(v('A') | v('B')) >> v('C')
-(v('A') & v('B'))
(-v('A')) ** v('B')
True
```

Parte III

Herramientas didácticas

Las funciones de este capítulo no intervienen en `test`; sirven para *ver* lo que `test` hace. `tabla_verdad` recorre todos los mundos posibles (el método de fuerza bruta de la sección 2), y `explica` muestra el árbol de refutación que construye `extension`. Ambas devuelven objetos que se imprimen como texto en la terminal y como HTML en un cuaderno Jupyter.

Capítulo 12

Átomos y subfórmulas

Para construir una tabla de verdad necesitamos la lista de átomos de una fórmula (las columnas de entrada) y, opcionalmente, la de sus subfórmulas compuestas (las columnas intermedias, ordenadas de dentro hacia fuera para que cada una se pueda calcular a partir de las anteriores). Ambas listas se devuelven sin repeticiones y en orden de aparición.

```
def atomos(P):
    """Proposiciones atómicas de ``P``, sin repetir, en orden de aparición."""
    if not isinstance(P, Proposicion):
        return []
    if not isinstance(P.data, list):
        return [P]
    res = []
    for x in P.data[1:]:
        for a in atomos(x):
            if a not in res:
                res.append(a)
    return res

def subformulas(P):
    """Subfórmulas compuestas de ``P`` (incluida ``P``), de dentro hacia fuera."""
    if not isinstance(P, Proposicion) or not isinstance(P.data, list):
        return []
    res = []
    for x in P.data[1:]:
        for s in subformulas(x):
            if s not in res:
                res.append(s)
    if P not in res:
        res.append(P)
    return res

_lista_subformulas = subformulas # alias: el parámetro de TablaVerdad se llama igual
```

Capítulo 13

Tablas de verdad

`tabla_verdad(P, premisas[], subformulas=False)` construye la tabla de verdad de P : una fila por cada uno de los 2^n mundos (n = número de átomos, contando también los de las premisas), con los átomos (en orden alfabético) en las primeras columnas, después (si se pide) las subfórmulas, después las premisas y por último P . Las filas se recorren en el orden habitual: todos los átomos V primero.

Si hay premisas, la tabla marca con $*$ las filas en que todas se cumplen: son los únicos mundos que importan para la consecuencia lógica. La propiedad `consecuencia` dice si P es V en todas esas filas, es decir, coincide con `test(P, premisas)` pero calculado por fuerza bruta. Sin premisas, las propiedades `tautologia` y `contradiccion` resumen la última columna.

`noDerivable` no tiene tabla de verdad (no depende del mundo, sino de las premisas), así que `span` lanzará un error si aparece.

```
class TablaVerdad:
    """Tabla de verdad de una fórmula, opcionalmente bajo unas premisas.

    Se construye con ``tabla_verdad(...)``. Atributos: ``atomos``,
    ``columnas`` (átomos, subfórmulas, premisas y la fórmula), ``filas``
    (lista de diccionarios columna -> bool), ``consecuencia``, ``tautologia``
    y ``contradiccion``. Se imprime como texto y, en Jupyter, como HTML.
    """

    def __init__(self, P, premisas=[], subformulas=False):
        if isinstance(P, bool):
            raise TypeError("tabla_verdad necesita una Proposicion, no un bool.")
        self.P = P
        self.premisas = [Q for Q in premisas if Q is not True]
        self._premise_falsa = any(Q is False for Q in self.premisas)
        self.premisas = [Q for Q in self.premisas if Q is not False]
        formulas = self.premisas + [P]
        conjunto = []
        for f in formulas:
            for a in atomos(f):
                if a not in conjunto:
                    conjunto.append(a)
        self.atomos = sorted(conjunto, key=repr)
        columnas = list(self.atomos)
        if subformulas:
            for f in formulas:
                for s in _lista_subformulas(f):
                    if s not in columnas and s not in formulas:
                        columnas.append(s)
        columnas += [Q for Q in self.premisas if Q not in columnas]
        columnas.append(P)
        self.columnas = columnas
        self.filas = []
        for valores in _itertools.product([True, False], repeat=len(self.atomos)):
            F = dict(zip(self.atomos, valores))
            self.filas.append({c: span(F, c) for c in columnas})

    def cumple_premisas(self, fila):
        """True si en esa fila todas las premisas son verdaderas."""
        return not self._premise_falsa and all(fila[Q] for Q in self.premisas)

    @property
    def consecuencia(self):
        """True si P es verdadera en todas las filas que cumplen las premisas."""
        return all(fila[self.P] for fila in self.filas if self.cumple_premisas(fila))

    @property
    def tautologia(self):
        return all(fila[self.P] for fila in self.filas)
```

```

@property
def contradiccion(self):
    return not any(fila[self.P] for fila in self.filas)

def resumen(self):
    """Frase que interpreta la tabla."""
    n = len(self.filas)
    if not self.premisas and not self.premisa_falsa:
        k = sum(fila[self.P] for fila in self.filas)
        if k == n:
            return f"{self.P!r} es V en las {n} filas: es una tautología "
                f"(test → True).")
        if k == 0:
            return f"{self.P!r} es F en las {n} filas: es una contradicción "
                f"(test → False).")
        return f"{self.P!r} es V en {k} de las {n} filas: es contingente, "
            f"no se deduce sin premisas (test → False).")
    marcadas = [i for i, fila in enumerate(self.filas, 1) if self.cumple_premisas(fila)]
    if not marcadas:
        return ("Ninguna fila cumple todas las premisas: son contradictorias entre "
            f"sí, y de una contradicción se deduce cualquier cosa (test → True).")
    malas = [i for i in marcadas if not self.filas[i - 1][self.P]]
    if not malas:
        donde = ("En la única fila marcada con *" if len(marcadas) == 1
            else f"En las {len(marcadas)} filas marcadas con *")
        return f"{donde} (premisas V), {self.P!r} es V: "
            f"se deduce de las premisas (test → True).")
    return f"En la fila {malas[0]} las premisas son V pero {self.P!r} es F: "
        f"es un contraejemplo, {self.P!r} no se deduce (test → False).")

def _cabeceras(self):
    return [repr(c) for c in self.columnas]

def __str__(self):
    cab = self._cabeceras()
    marca = bool(self.premisas) or self.premisa_falsa
    anchos = [max(len(h), 1) for h in cab]
    lineas = []
    if marca:
        lineas.append(" " + " | ".join(h.center(w) for h, w in zip(cab, anchos)))
    else:
        lineas.append(" | ".join(h.center(w) for h, w in zip(cab, anchos)))
    lineas.append(("---" if marca else "") + "-+-".join("-" * w for w in anchos))
    for fila in self.filas:
        celdas = [_VF(fila[c]).center(w) for c, w in zip(self.columnas, anchos)]
        pref = (" * " if self.cumple_premisas(fila) else " ") if marca else ""
        lineas.append(pref + " | ".join(celdas))
    lineas.append("")
    lineas.append(self.resumen())
    return "\n".join(lineas)

__repr__ = __str__

def _repr_html_(self):
    cab = self._cabeceras()
    marca = bool(self.premisas) or self.premisa_falsa
    esc = _html.escape
    h = ['<table style="border-collapse:collapse;font-family:monospace">',
        '<thead><tr>']
    if marca:
        h.append('<th></th>')
    for c, texto in zip(self.columnas, cab):
        estilo = "border-bottom:2px solid #888;padding:2px 8px"
        if c == self.P:
            estilo += ";border-left:2px solid #888"
        h.append(f'<th style="{estilo}">{esc(texto)}</th>')
    h.append('</tr></thead><tbody>')
    for fila in self.filas:
        ok = self.cumple_premisas(fila)
        fondo = "background:#fff3c4" if (marca and ok) else ""
        h.append(f'<tr style="{fondo}">')
        if marca:
            h.append(f'<td style="padding:2px 6px">{"*" if ok else ""}</td>')
        for c in self.columnas:
            estilo = "text-align:center;padding:2px 8px"
            if c == self.P:
                estilo += ";border-left:2px solid #888;font-weight:bold"
                if marca and ok and not fila[c]:
                    estilo += ";color:#c00"
            h.append(f'<td style="{estilo}">{_VF(fila[c])}</td>')
        h.append('</tr>')
    h.append('</tbody></table>')
    h.append(f'<p style="font-family:monospace">{esc(self.resumen())}</p>')
    return "".join(h)

```

```
def tabla_verdad(P, premisas=[], subformulas=False):
    """Tabla de verdad de ``P`` (bajo ``premisas`` si se dan); véase ``TablaVerdad``.

    Con ``subformulas=True`` se añaden columnas para cada subfórmula, de dentro
    hacia fuera, como en las tablas que se construyen a mano.
    """
    return TablaVerdad(P, premisas, subformulas)
```

```
print( tabla_verdad(v('B'), [v('A'), v('A') >> v('B')]) )
```

	v('A')		v('B')		v('A') >> v('B')		v('B')
*	V		V		V		V
	V		F		F		F
	F		V		V		V
	F		F		V		F

En la única fila marcada con * (premisas V), v('B') es V: se deduce de las premisas (test → True).

Capítulo 14

Explicación del razonamiento

`explica(P, premisas[])` ejecuta refuta con una traza y devuelve un objeto `Razonamiento` que, al imprimirse, muestra:

1. la pregunta (¿se deduce P de las premisas?) y el método (buscar un contraejemplo);
2. el árbol de refutación paso a paso, con las reglas aplicadas y las ramas que se cierran;
3. la conclusión: o bien el contraejemplo encontrado (y `test` vale `False`), o bien que todas las ramas se cierran (y `test` vale `True`).

El atributo `resultado` es el valor de `test(P, premisas)` y `contraejemplo` es el modelo encontrado (o `None`).

```
class Razonamiento:
    """Taza del razonamiento de ``test``; se construye con ``explica(...)``.

    Atributos: ``P``, ``premisas``, ``pasos`` (lista de líneas), ``resultado``
    (el valor de ``test``) y ``contraejemplo`` (diccionario o ``None``).
    """
    def __init__(self, P, premisas, pasos, resultado, contraejemplo):
        self.P, self.premisas, self.pasos = P, premisas, pasos
        self.resultado, self.contraejemplo = resultado, contraejemplo

    def _cabecera(self):
        if self.premisas:
            prem = ", ".join(repr(Q) for Q in self.premisas)
            return [f"¿Se deduce {self.P!r} de las premisas {prem}?",
                    "Método: buscamos un contraejemplo, es decir, un mundo en el que "
                    "las premisas sean V y la fórmula F. Si toda rama se cierra por "
                    "contradicción, no existe tal mundo y la fórmula se deduce."]
            return [f"¿Es {self.P!r} una tautología (se deduce sin premisas)?",
                    "Método: buscamos un contraejemplo, es decir, un mundo en el que la "
                    "fórmula sea F. Si toda rama se cierra por contradicción, no existe "
                    "tal mundo y la fórmula es una tautología."]

    def _conclusion(self):
        if isinstance(self.P, bool):
            return [f"La fórmula es directamente {self.P}: test → {self.resultado}."]
        if self.resultado:
            return ["Conclusión: todas las ramas se cierran, no hay contraejemplo. "
                    f"{self.P!r} se deduce de las premisas: test → True."]
        return [f"Conclusión: el mundo {fmt_modelo(self.contraejemplo)} es un "
                f"contraejemplo (los átomos no listados pueden tomar cualquier valor). "
                f"{self.P!r} no se deduce de las premisas: test → False."]

    def __str__(self):
        if isinstance(self.P, bool):
            return "\n".join(self._conclusion())
        return "\n".join(self._cabecera() + [""] + self.pasos + [""] + self._conclusion())

    __repr__ = __str__

    def _repr_html_(self):
        esc = _html.escape
        if isinstance(self.P, bool):
            return f'<p style="font-family:monospace">{esc(self._conclusion()[0])}</p>'
        cab = "".join(f"<p>{esc(l)}</p>" for l in self._cabecera())
        pasos = []
        for linea in self.pasos:
            color = "#080" if "✓" in linea else ("c00" if "x" in linea else "")
            return f'<p style="font-family:monospace">{cab}</p>{esc(linea)}</p>'
```

```

        pasos.append(f'<span style="color:{color}">{esc(linea)}</span>')
    fin = self._conclusion()[0]
    color = "#000" if self.resultado else "#c00"
    return (f'<div style="font-family:monospace">{cab}</div>'
            f'<pre style="margin:6px 0 6px 1em">{"<br>".join(pasos)}</pre>'
            f'<p style="color:{color}"><b>{esc(fin)}</b></p></div>')

def explica(P, premisas=[]):
    """Razonamiento paso a paso de ``test(P, premisas)``; véase ``Razonamiento``."""
    if isinstance(P, bool):
        return Razonamiento(P, list(premisas), [], P, None)
    pasos = []
    M = refuta(P, premisas, pasos)
    resultado = not M
    contraejemplo = None if resultado else {a: b for a, b in M.items() if not _es_pseudoatomo(a)}
    return Razonamiento(P, [Q for Q in premisas if not isinstance(Q, bool)],
                        pasos, resultado, contraejemplo)

print( explica(v('B'), [v('A'), v('A') >> v('B')]) )

```

¿Se deduce $v('B')$ de las premisas $v('A')$, $v('A') \gg v('B')$?

Método: buscamos un contraejemplo, es decir, un mundo en el que las premisas sean V y la fórmula F. Si toda ram

Buscamos un mundo en el que $v('B') = F$, $v('A') = V$, $v('A') \gg v('B') = V$:

$v('B') = F \rightarrow$ anotamos $v('B') = F$.

$v('A') = V \rightarrow$ anotamos $v('A') = V$.

$v('A') \gg v('B') = V \Rightarrow$ por la regla de $\gg$: o bien $v('A') = F$, o bien $v('B') = V$

Rama 1 de 2: suponemos $v('A') = F$

$v('A') = F$ $\times$ contradice lo anotado ($v('A') = V$): rama cerrada.

Rama 2 de 2: suponemos $v('B') = V$

$v('B') = V$ $\times$ contradice lo anotado ($v('B') = F$): rama cerrada.

$\times$ Las 2 ramas se cierran.

Conclusión: todas las ramas se cierran, no hay contraejemplo. $v('B')$ se deduce de las premisas: $\text{test} \rightarrow \text{True}$.